

Evaluation Flow for the Web-Search RAG QA System

This document defines the evaluation plan for our Question Answering Agents project. The system is a web-search-based RAG / multi-agent QA pipeline using Tavily for live web retrieval and optional agentic components such as query decomposition, HyDE, hybrid retrieval, reranking, evidence filtering, answer writing, and a Self-RAG critic loop.

Important privacy note: benchmark examples that contain canary strings or dataset-internal content should not be copied into reports, public repositories, screenshots, or training data. Only aggregate metrics and non-sensitive descriptions should be published.

1. Evaluation Goal

The evaluation measures whether the system answers factual questions correctly, whether retrieval improves over an LLM-only baseline, which pipeline components add value, whether selected evidence supports the answer, and whether the system avoids hallucinations by abstaining when evidence is insufficient.

We explicitly distinguish two retrieval settings: live-web evaluation with Tavily, and controlled RAG evaluation with CRAG mock APIs.

2. Benchmarks

SimpleQA is used as a live-web diagnostic benchmark. It contains short factual questions with unambiguous answers and is well suited for debugging the Tavily-based pipeline. We sample about 150-200 questions and evaluate them with Tavily.

CRAG is used as a controlled RAG benchmark, not as a Tavily benchmark. CRAG provides question-answer pairs plus mock web and knowledge graph APIs. For CRAG, Tavily should be replaced by a CragSearcher adapter so that the downstream RAG pipeline is evaluated in a controlled retrieval setting. We sample about 50-100 temporally stable questions.

BrowseComp is used only as a stretch test for hard agentic web search. It is much harder than SimpleQA or CRAG and should be evaluated with Tavily on a small subset of about 20-30 questions.

3. Retrieval Provider Abstraction

The pipeline should use a common SearchProvider interface: `search(query) -> list[SearchResult]`.

TavilySearcher implements live web search for SimpleQA and BrowseComp.

CragSearcher implements access to CRAG mock web / knowledge graph APIs for controlled CRAG evaluation.

Only the retrieval provider changes between benchmarks; the downstream RAG pipeline remains the same.

4. System Setups

S0 LLM-only: no retrieval, only a direct model answer.

S1 Basic RAG: search provider, chunking, simple lexical scoring, answer writer, critic.

S2 Query Decomposition: S1 plus query planning and multiple focused search queries.

S3 Hybrid Retrieval + Reranking: S2 plus BM25, dense embeddings, and cross-encoder

reranking.

S4 Evidence Filtering: S3 plus LLM-based evidence selection with reasons.

S5 Full System: query decomposition, HyDE, hybrid retrieval, reranking, evidence filtering, answer writer, and Self-RAG critic loop.

For BrowseComp, only S1 and S5 should be evaluated to keep cost and runtime manageable.

5. Primary Metric

The primary metric is Truthfulness Score, adapted from CRAG's evaluation scheme.

Perfect / correct / acceptable answer: +1.

Missing / not attempted answer: 0.

Incorrect / hallucinated answer: -1.

Truthfulness is the average score over all questions. This metric rewards correct answers, penalizes hallucinations, and treats abstention as better than unsupported guessing.

6. Secondary Metrics

Accuracy: share of correct or acceptable answers.

Missing Rate: share of missing or not attempted answers.

Hallucination Rate: share of incorrect answers.

Average Latency: mean runtime per question.

Average Search Calls: number of Tavily or CRAG API calls.

Average Sources and Evidence Chunks: how much context the system used.

Evidence Relevance: 2 means directly supports the answer, 1 means partially useful, 0 means irrelevant.

Citation Support Rate: share of answers where cited evidence actually supports the central answer.

7. Automatic Evaluation

Each generated answer is first checked by normalized exact match against the gold answer. If exact match does not decide the case, an LLM judge labels the answer.

The judge receives the question, gold answer, system answer, and selected evidence. It returns JSON with label, truthfulness_score, evidence_relevance, and a short reason.

The judge model should ideally differ from the answer writer model. If this is not possible, it must be reported as a limitation.

8. Human Evaluation

A small human evaluation validates whether automatic metrics are meaningful. Annotate about 40 examples, for example 20 from SimpleQA, 15 from CRAG, and 5 from BrowseComp.

Human labels: answer correctness, groundedness, and evidence relevance.

Report agreement between human labels and automatic judge labels. If possible, report Spearman correlation or Cohen's kappa when two annotators are available.

9. Ablation Study

S0 to S1 measures the value of retrieval.

S1 to S2 measures the value of query decomposition.

S2 to S3 measures the value of hybrid retrieval and reranking.

S3 to S4 measures the value of the evidence filtering agent.

S4 to S5 measures the value of HyDE and the Self-RAG critic loop.

For the critic loop, log whether a revision happened, whether the revision improved the score, and how much latency it added.

10. Per-Benchmark Flow

SimpleQA: SimpleQA question -> TavilySearcher -> RAG pipeline -> answer -> gold answer comparison -> judge -> JSONL result.

CRAG: CRAG question -> CragSearcher using CRAG mock APIs -> same RAG pipeline -> CRAG-style auto evaluation -> JSONL result. No Tavily calls should be used in CRAG evaluation.

BrowseComp: BrowseComp question -> TavilySearcher -> RAG pipeline -> answer -> gold answer / judge evaluation -> JSONL result.

11. JSONL Output Schema

Each run should store: benchmark, retrieval_provider, question_id, setup, question, gold_answer, prediction, judge_label, truthfulness_score, evidence_relevance, latency_seconds, search_calls, sources_count, evidence_count, critic_needs_revision, critic_comment, and selected_evidence metadata.

12. Result Tables

Dataset Usage: benchmark, full size, used sample, retrieval provider, purpose.

SimpleQA Results: setup, truthfulness, accuracy, missing, hallucination, latency, search calls.

CRAG Results: setup, truthfulness, accuracy, missing, hallucination, evidence relevance, latency.

BrowseComp Stress Test: setup, truthfulness, accuracy, missing, hallucination, latency.

Evidence Quality: benchmark, setup, evidence relevance, citation support, average evidence chunks.

Human vs Automatic Evaluation: correctness agreement, groundedness agreement, evidence relevance agreement or correlation.

13. Suggested Minimal Version

If time or budget is limited, run SimpleQA with 100 questions and setups S1, S3, S5; CRAG with 30 stable questions and setups S1, S5 using CRAG mock APIs; and BrowseComp with 10 questions and setups S1, S5. This gives about 380 runs and is realistic for a university project.

14. Report Wording

We evaluate our system in three complementary settings. First, SimpleQA is used as a diagnostic benchmark for live-web factual question answering with Tavily. Second, a temporally stable CRAG subset is used as a controlled RAG benchmark. Since CRAG provides mock web and knowledge graph APIs, Tavily is replaced with a CRAG retrieval adapter for these runs. Third, a small BrowseComp subset is used as a stress test for hard agentic web search.

Our primary metric is Truthfulness Score, adapted from CRAG: correct or acceptable answers receive +1, missing answers receive 0, and incorrect or hallucinated answers receive -1. We additionally report accuracy, missing rate, hallucination rate, latency, search calls, and evidence relevance. A small human evaluation validates the automatic judge.

Sources

OpenAI SimpleQA: <https://openai.com/index/introducing-simpleqa/>

OpenAI BrowseComp: <https://openai.com/index/browsecomp/>

CRAG paper: <https://arxiv.org/abs/2406.04744>

CRAG / Aicrowd challenge: <https://www.aicrowd.com/challenges/meta-comprehensive-rag-benchmark-kdd-cup-2024>